MODULE I

Module 1: Effective Software Development: Principles and Best Practices

Definition and Characteristics of Software Engineering. Phases of the Software Lifecycle: Design, Development, Testing, Deployment, Maintenance.

Project Planning: Objectives and Scope, Estimation Models (e.g., COCOMO). Software Engineering Models: Predictive Models, Adaptive Models, Waterfall Variants, Prototyping.

Software Engineering

- ✔ Software engineering is an engineering discipline that is concerned with all aspects of software production.
- ✔ That is from the early stages of system specification to maintaining the system after it has gone into use.
- ✔ Software engineering is the establishment and use of engineering principles in order to obtain economically feasible software that is reliable and works efficiently on real machines.

Software Engineering

- **Software engineering is "a systematic approach to the analysis, design, assessment, implementation, test, maintenance and reengineering of software, that is, the application of engineering to software"**

- Software engineering is the establishment and use of engineering principles in order to obtain economically feasible software that is reliable and works efficiently on real machines.

- Software engineering encompasses all aspects of conceiving, communicating, specifying, designing, building, testing, and maintaining software systems.

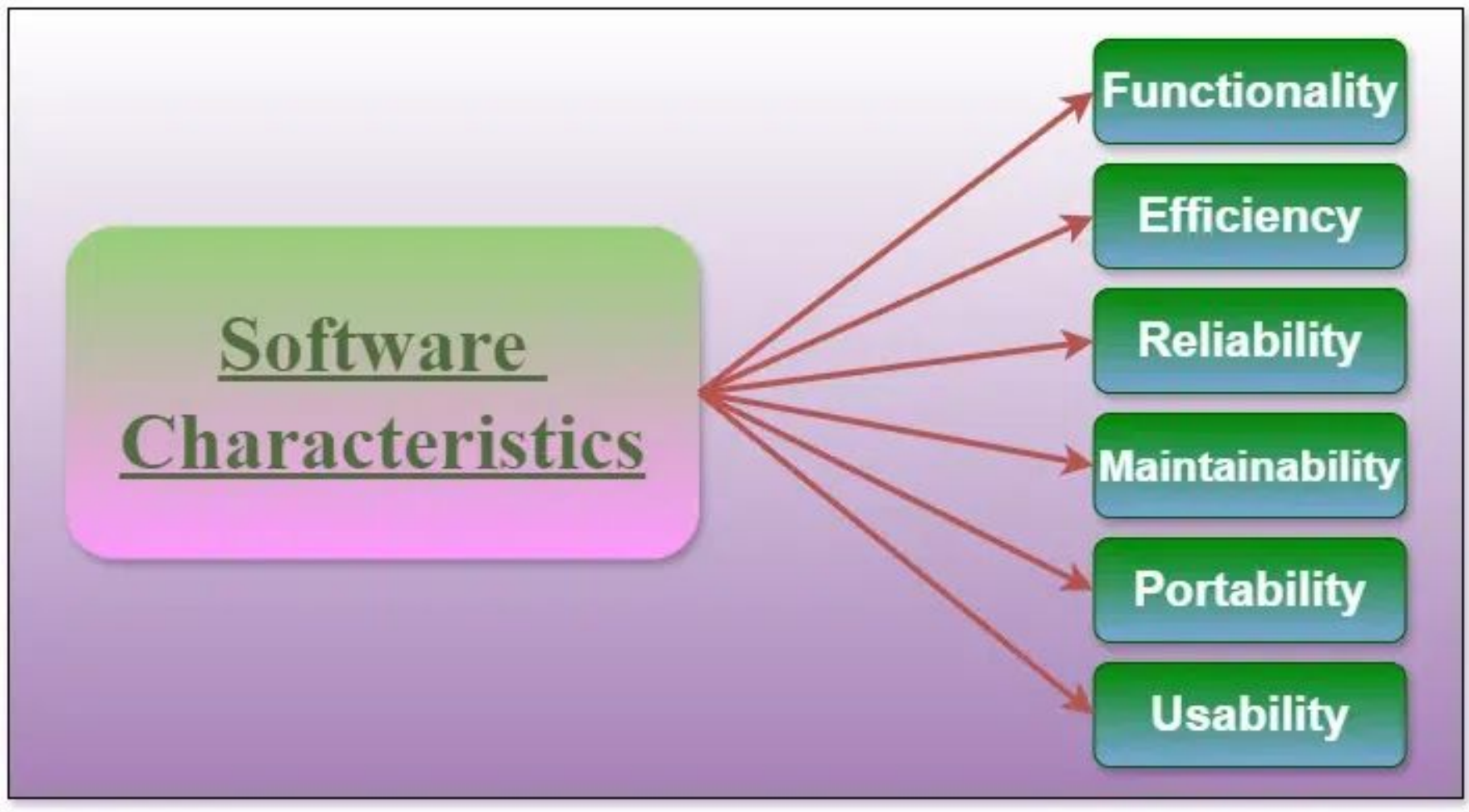
Why Learn Software Engineering?

- ✔ Software engineers play an important role in today's information driven society.
- ✔ Software engineering is one of the fastest growing professions with excellent job prospects predicted throughout the coming decade.
- ✔ Software engineering brings together various skills and responsibilities.
- ✔ Studying software engineering opens up a wide range of career opportunities like Software Developer, Test Engineer, Project Manager, Software Architect, Software Quality Manager, Doctoral Student/Scientist
- ✔ Software engineers probably spend less than 10% of their time writing code.
- ✔ The other 90% of their time is involved with other activities that are more important than writing code.
- ✔ These activities include: Modeling and Optimization

Characteristics of Software Engineering

- ✔ Software can be characterized by external qualities and internal qualities.
- ✔ External qualities, such as usability and reliability, are visible to the user.
- ✔ Internal qualities are those that may not be necessarily visible to the user. Eg: good requirements and design documentation.
- ✔ Software engineering's main characteristics include
  - ✔ functionality,
  - ✔ reliability,
  - ✔ efficiency,
  - ✔ usability,
  - ✔ maintainability,
  - ✔ and portability

Characteristics of Software Engineering



Characteristics of Software

1. **Functionality**

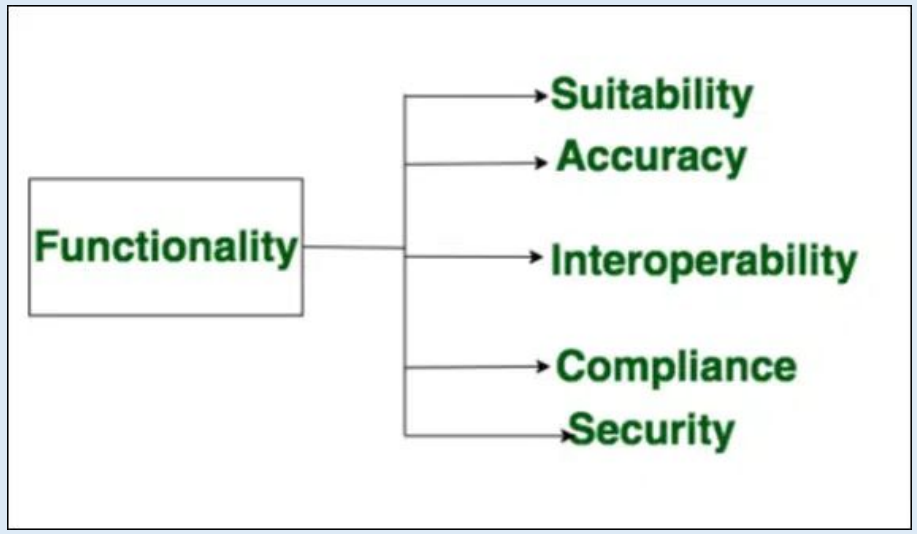
   Functionality refers to the set of features and capabilities that a software program or system provides to its users.

   The extent to which the software provides functions that meet stated and implied needs.

   It determines the usefulness of the software for the intended purpose.

   Examples of functionality in software include: Required Functions are

- ✔ Data storage and retrieval
- ✔ Data processing and manipulation
- ✔ User interface and navigation
- ✔ Communication and networking
- ✔ Security and access control
- ✔ Reporting and visualization
- ✔ Automation and scripting



Characteristics of Software

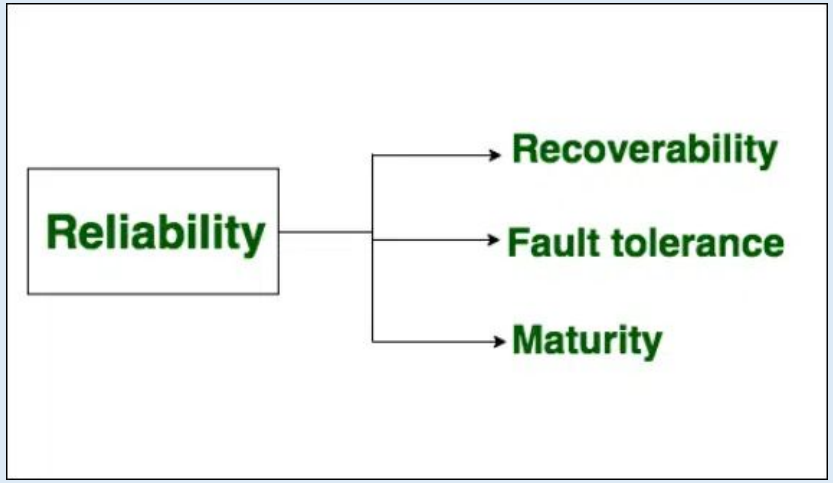
2. Reliability

Reliability is a characteristic of software that refers to its ability to perform its intended functions correctly and consistently over time.

Reliability helps ensure that the software will work correctly and not fail unexpectedly.

Examples of factors that can affect the reliability of software include:

1. Bugs and errors in the code
2. Lack of testing and validation
3. Poorly designed algorithms and data structures
4. Inadequate error handling and recovery
5. Incompatibilities with other software or hardware



To improve the reliability of software, various techniques, and methodologies can be used, such as testing and validation, formal verification etc.

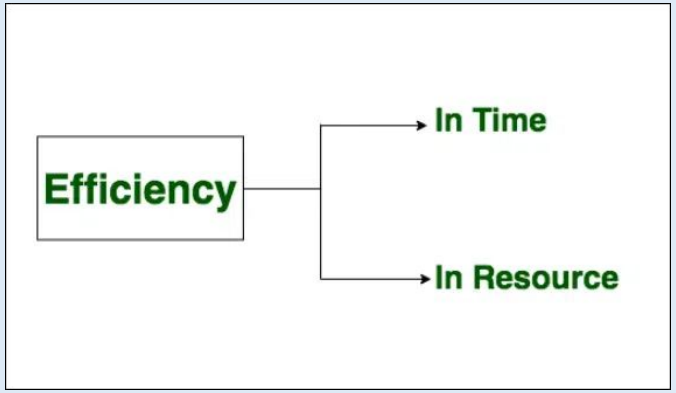
Characteristics of Software

3. Efficiency

It refers to the ability of the software to use system resources in the most effective and efficient manner. Efficiency is a characteristic of software that refers to its ability to use resources such as memory, processing power, and network bandwidth in an optimal way. High efficiency means that a software program can perform its intended functions quickly and with minimal use of resources, while low efficiency means that a software program may be slow or consume excessive resources.

Examples of factors that can affect the efficiency of the software include:

- ✔ Poorly designed algorithms and data structures
- ✔ Inefficient use of memory and processing power
- ✔ High network latency or bandwidth usage
- ✔ Unnecessary processing or computation
- ✔ Unoptimized code



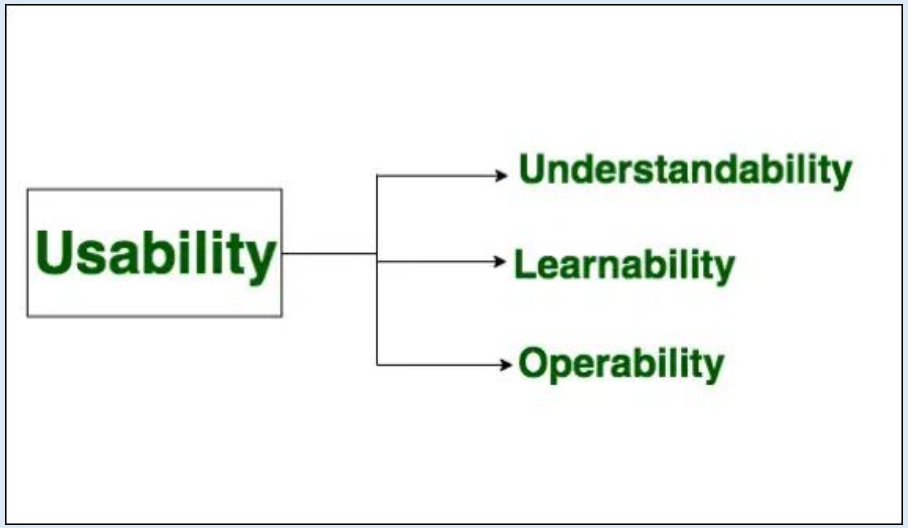
To improve the efficiency of software, various techniques, and methodologies can be used, such as performance analysis, optimization, and profiling.

Characteristics of Software

4. Usability

It refers to the extent to which the software can be used with ease. the amount of effort or time required to learn how to use the software.

Required functions are:

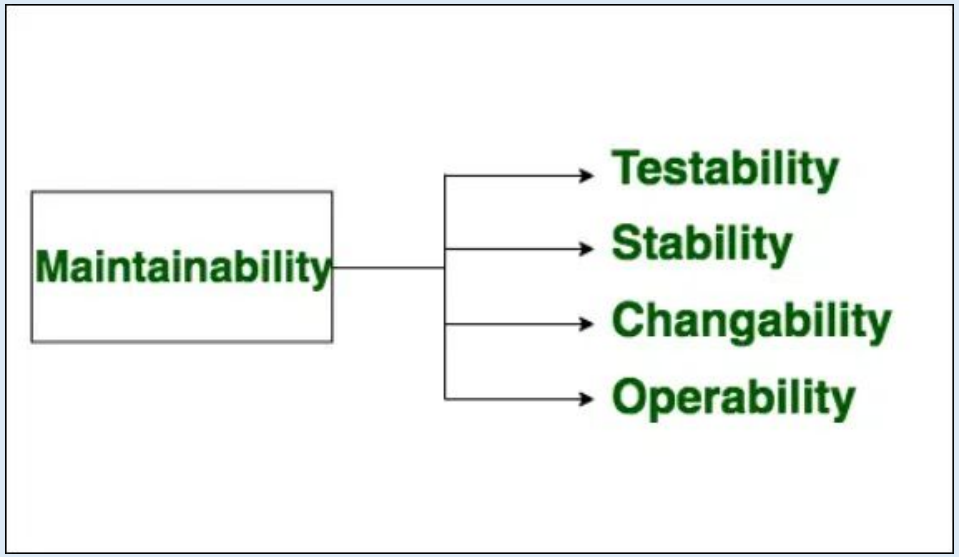


Characteristics of Software

5. Maintainability

It refers to the ease with which modifications can be made in a software system to extend its functionality, improve its performance, or correct errors.

Required functions are:

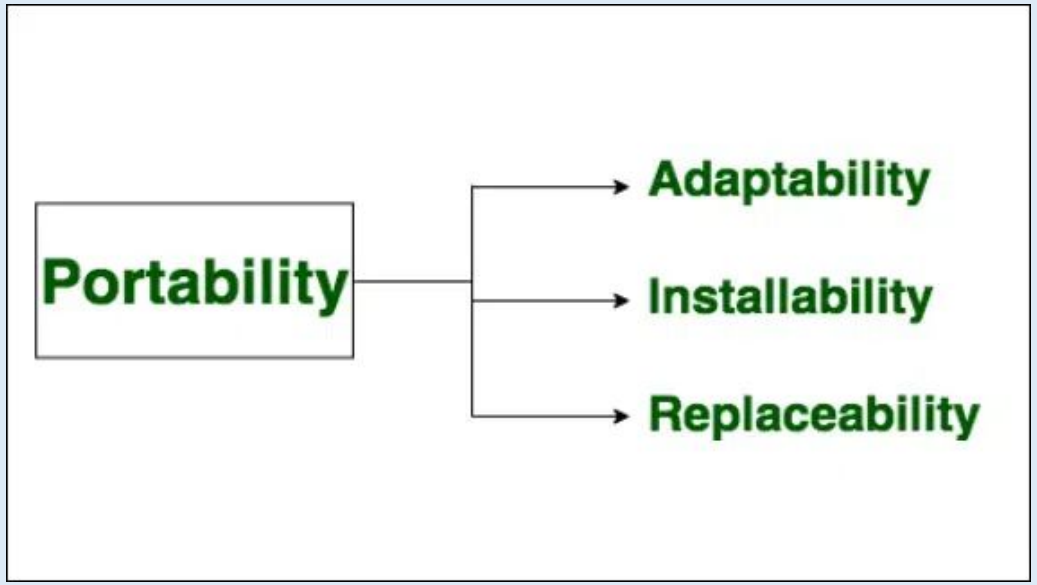


Characteristics of Software

6. Portability

A set of attributes that bears on the ability of software to be transferred from one environment to another, without minimum changes.

Required functions are:



Life cycle of a software system

6 Phases of the Software Development Life Cycle


4
QUALITY ASSURANCE
5
DEPLOYMENT
3
IMPLEMENTATION
THE
SOFTWARE
DEVELOPMENT
CYCLE
6
SUPPORT &
MAINTENANCE
2
DESIGN
1
ANALYSIS

SDLC Phase 1: Requirement Analysis & Planning

- Phase conduct by Product manager, Business Analyst or senior members in team.
- To understand the Aim, Purpose & Exact requirements of the customer.
- Conduct market research, customer interviews & surveys.
- Software Requirement Specification (SRS) document is created.
- Identify risk assessment in project.
- Planning of Project schedule, Cost estimation & other recourses.

Example: Online Shopping Application

Customer requirements like:

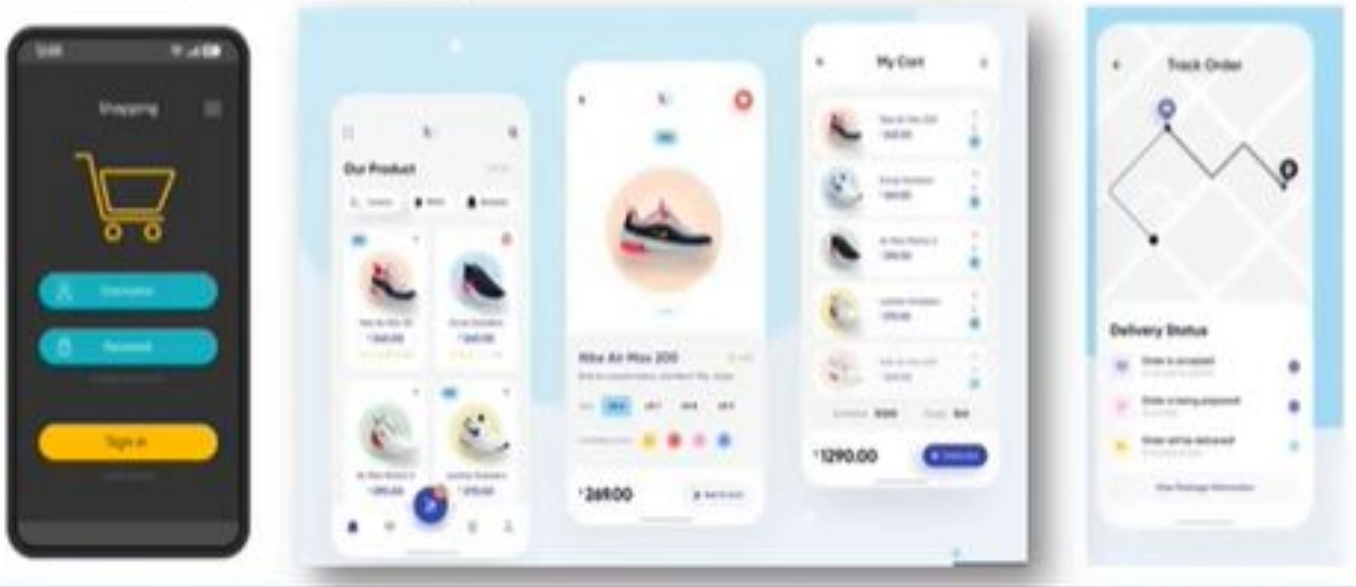
- Categories like Ladies wear, Gents wear module, Grocery module, Shopping cart module, My order module, Discount module, Notification module.
- Good & Attractive user interface.

SDLC Phase 2: Design

- Phase conduct by Product manager & UI / UX Designer team.
- High level design include Software Architecture like brief description of each module, Flow Diagram, Database tables, Interface relationships & Algorithm.
- Low level design include Input & Output of each module, Rough paper design, Detail of Interface, Listing error messages etc.
- Documented in DDS "Design Document Specification".

Example:

Online Shopping Application



SDLC Phase 3: Implementation

- Phase conduct by Developer or Programmer.
- Must follow predefine coding guidelines or discuss with management team.
- Used suitable Programming Languages like C, C++, JAVA, ANDROID, PYTHON, PHP etc.
- Used suitable Databases like SQL, ORACLE etc.
- Programming tools like Compilers, Interpreters, Debuggers etc.

Example:

Online Shopping Application

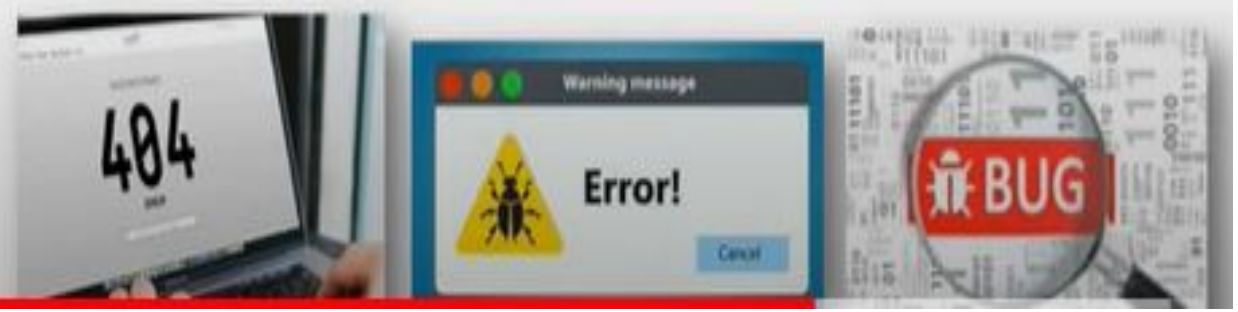
```
void customerDetails()
{
    cout << "Enter your name: ";
    string customer_name;
    getline(cin,customer_name );
    cout << "WELCOME ";
    for(int i = 0; i<customer_name.length(); i++ )
    {
        cout << char(toupper(customer_name[i] ));
    }
    cout << "\n";
}
```

SDLC Phase 4: Testing

- Phase conduct by QA Team or Tester.
- It perform Integration Testing, Black Box Testing, White Box Testing, System Testing, Regression Testing Performance Testing & Acceptance Testing etc.
- The testing team starts testing the functionality of the entire system.
- If find some bugs/defects which report to developers then Development team fixes the bug and send back to QA.
- This is done to verify that the entire application works according to the customer requirement.

Example:

Online Shopping Application



SDLC Phase 5: Deployment & Maintenance

- Phase conduct by Project Manager, Operation or Production support team.
- Once the software is certified and no bugs or errors are stated, then final software is release in market or deployed in customer environment.
- If any issue comes up and needs to be fixed or any updating is to be done is taken care by the Support Managers in Maintenance phase.

Example:

Online Shopping Application

Project Planning Phase

Project objectives, Scope of the software system, Empirical estimation models-COCOMO

Project Planning Phase

Project Planning is the phase in software engineering where the project's **objectives, scope, resources, schedule, cost, and risks** are defined before actual development begins.

It acts as a **roadmap** that guides the project team in carrying out the software development activities efficiently.

Project objectives

- Project objectives are specific, measurable goals that outline what the project aims to achieve.
- Defining clear objectives helps guide the project team and provides benchmarks for measuring success.
- Here's how to structure project objectives:

Specific:

- Clearly state what the project will accomplish. Avoid vague language.
- Example: Develop a web-based Customer Relationship Management (CRM) system to streamline client interactions and improve sales tracking.

Project objectives

Measurable:

- Establish criteria to assess progress and success. Define how success will be measured.
- Example: Achieve a 25% increase in sales conversion rates within the first six months of system implementation

Achievable:

- Ensure the objectives are realistic and attainable, considering available resources and constraints.
- Example: Deliver the CRM system within an allocated budget of $150,000 and a project timeline of 8 months.

Project objectives

Time-bound:

- Set a clear timeline for achieving the objectives, including milestones and deadlines.
- Example: Complete user training and go live with the CRM system by the end of Q3 2025.

Scope of the software system

- ✔ The **scope of a software system** defines the **boundaries of the problem** to be solved and sets clear limits on what the system will and will not do. It is the foundation of project planning and requirement analysis.

  Software Scope identifies:

  - ✔ The **functions and features** to be delivered.
  - ✔ The **data** the software will process.
  - ✔ The **end users** who will interact with the system.
  - ✔ The **constraints** under which the system will operate.

  It provides a **bounded statement of the problem**, preventing “scope creep” (uncontrolled growth of requirements).

Scope of the software system

- Key Elements of Software Scope

Product Function

- Describes **what the software will do**.
- Defines major system capabilities.
- Ex:Online banking system may include account management, fund transfers & bill payments functions.

Performance Requirements

- Defines the **quality attributes** such as speed, reliability, security, and usability.
- Ex: “The system must support 10,000 concurrent users.”

Constraints

- Limitations that affect development and use.
- Ex: Hardware restrictions, regulatory policies, budget, deadlines

Interfaces

- Specifies how the system will interact with other software, hardware, or users.
- Ex: ATM software must interface with bank databases and network services.

Scope of the software system

Key elements of the scope include

Inclusions:

- Specify the features, functionalities, and components that will be part of the software system.

Example:

- User account management (registration, login, password recovery)
- Lead tracking and management
- Sales forecasting and reporting
- Integration with existing marketing automation tools
- Dashboard for real-time analytics and insights

Scope of the software system

Exclusions:

- Clearly outline what will not be included in the project to manage expectations.

Example:

- No mobile application development at launch

Stakeholder Requirements:

- Document the specific needs and requirements of stakeholders that the software must fulfill.

Example

Sales team requires a streamlined process for managing leads, while the marketing team needs comprehensive reporting capabilities.

Scope of the software system

Scope is defined using one of two techniques:

1. A narrative description of software scope is developed after communication with all stakeholders.

2. A set of use cases is developed by end users

- ✔ Performance considerations encompass processing and response time requirements.
- ✔ Constraints identify limits placed on the software by external hardware, available memory, or other existing systems.
- ✔ **Once scope has been identified (with the concurrence of the customer), it is reasonable to ask:**
  - "Can we build software to meet this scope?
  - Is the project feasible?"

Project Estimation

The main task involved in **project planning** is project estimation

Following project attributes are estimated

Cost : How much it is going to cost to develop the software product?

Effort: How much effort would be necessary to develop the product?

Duration: How long it is going to take to develop the product?

Importance of Accurate Estimation

The quality of project plan depends on the accuracy of estimates

Parameters considered are:

- Project size
- Effort required to complete the project
- Project duration
- Cost

Accuracy of estimates is important

- They help in quoting appropriate project cost to customer
- Forms the basis for resource planning & scheduling

Cost Estimation Models

Cost estimation models are broadly classified into two categories:

(a) Algorithmic Models

Use mathematical equations based on historical data or theoretical models. Provide more quantitative and repeatable estimates.

Examples:

COCOMO (Constructive Cost Model)

COCOMO II

Software Equation

(b) Expert Judgment (Non-Algorithmic Models)

Relies on the experience and intuition of project managers, experts, or historical project data.

Involves techniques like:

Delphi technique (multiple experts give estimates anonymously, then results are averaged/refined).

Analogy-based estimation (comparing with similar past projects).

Useful when historical data is limited or project is unique.

COCOMO MODEL

- The **COCOMO (Constructive Cost Model)** is a widely used algorithmic model for estimating the **cost, effort, and time** required to develop a software project.
- It was developed by **Barry Boehm** in 1981 and has evolved into various versions like **COCOMO II** to handle modern software development methodologies.

Key Components of the COCOMO Model

1. Project Types:

✔ COCOMO defines three types of projects, each with different complexity and constants:

- **Organic**: Simple projects with small teams and well-understood requirements (e.g., business applications).
  - Example: A payroll system for a small company.
- **Semi-Detached**: Medium-complexity projects with more diverse teams and moderately understood requirements (e.g., complex database systems).
  - Example: A mid-sized CRM system.
- **Embedded**: Complex projects with real-time systems, hardware integration, or strict constraints (e.g., defense systems).
  - Example: An air traffic control system.

Key Components of the COCOMO Model

2.Effort Estimation (E):

- ✔ The COCOMO model estimates the total **effort (E)** required to complete the project in **person-months (PM).**
- ✔ This is done using a formula that depends on the size of the project (measured in thousands of lines of code, or **KLOC**) and constants based on the project type.

$$E = a \cdot (KLOC)^b$$

where

- **E** is the effort (person-months).
- **a** and **b** are constants that depend on the type of project.
- **KLOC** is the estimated size of the software (in thousands of lines of code).

Key Components of the COCOMO Model

3.Development Time (T/D):

The time required to complete the project in months, calculated as:

$$T = c \cdot (E)^d$$

where

- **T** is the development time in months.
- **c** and **d** are constants based on the project type.
- **E** is the effort calculated above.

Key Components of the COCOMO Model

Here are the constant values for the Basic COCOMO model, categorized by the three development modes (Organic, Semi-detached, and Embedded)

Basic COCOMO coefficients

Software Project	a	b	c	d
Organic	2.4	1.05	2.5	0.38
Semidetached	3.0	1.12	2.5	0.35
Embedded	3.6	1.20	2.5	0.32

Advantage And Disadvantage of the COCOMO Model

Advantages of COCOMO

- Easy to understand and apply.
- Provides a good initial estimation.
- Adaptable to different types of projects.

Disadvantages of COCOMO

- Depends heavily on accurate estimation of **KLOC**, which can be difficult in early stages.
- It may not account well for modern agile or iterative development methodologies.

Example Calculation

For an **organic project** with an estimated **size of 30 KLOC**:

- The constants for an organic project are: $a = 2.4$, $b = 1.05$, $c = 2.5$, and $d = 0.38$.
- The effort (E) would be:

$$E = 2.4 \cdot (30)^{1.05} \approx 91.5 \, \text{person-months}$$

- The development time (T) would be:

$$T = 2.5 \cdot (91.5)^{0.38} \approx 13.2 \, \text{months}$$

This means it will take about **91.5 person-months** of effort and **13.2 months** of time to develop the project.

Suppose that a project was estimated to be 400 KLOC. Calculate the effort and development time for each of the three modes i.e., organic, semi detached and embedded.

- Effort $(E) = a \cdot (KLOC)^b$ PM (Person-Months)
- Development Time $(D) = c \cdot (E)^d$ Months

Here, $KLOC = 400$. The constants a, b, c, d vary for each mode.

Software Engineering models

Predictive Models, Adaptive Models, Waterfall Variants, Prototyping

Software Engineering models

Predictive Model

- Predictive development model Predict in advance what needs to be done and then do it.
- It uses the requirements to design the system, and use the design as a blueprint to write the code and then test the code.
- **Waterfall Model:** A linear approach where each phase must be completed before moving on to the next. It works best for projects with well-defined requirements.
- **V-Model:** An extension of the waterfall model that emphasizes validation and verification at each development stage. It follows the same linear steps as Waterfall, but testing activities are planned in parallel with corresponding development stages.

Software Engineering models

Predictive Model

- Disadvantages:
- Inflexible: If the requirements change, it is very hard to implement.
- Later initial release: Many adaptive models enables us to give the users a program as soon as it does something useful. With a predictive model, the users don't get anything until development is finished.
- Big Design Up Front (BDUF): we need to define everything up front. We can't start development until we know exactly everything what we are going to do.

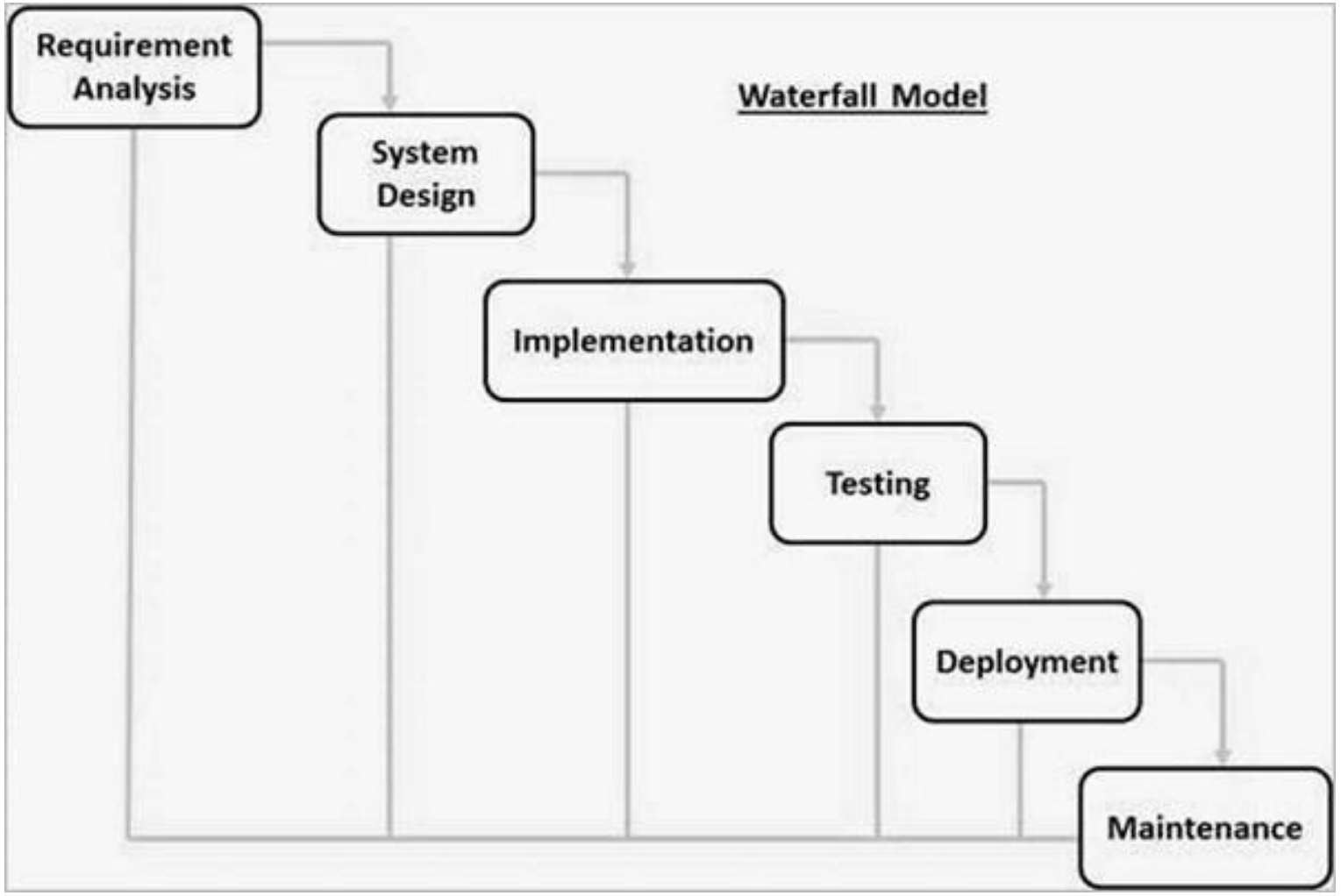
Predictive Software Engineering Models

- ✔ Waterfall
- ✔ Waterfall with Feedback
- ✔ Incremental Waterfall
- ✔ Sashimi
- ✔ V Model

Waterfall Model

- *Waterfall* is the plain vanilla(simplest version) of the predictive model world.
- It follows a linear approach where each phase (requirements, design, implementation, testing, deployment, and maintenance) must be completed before the next one begins.
- It assumes that you finish each step completely and thoroughly before you move on to the next step.
- The waterfall model can work reasonably well if all the following assumptions are satisfied:
  - ✔ The requirements are precisely known in advance.
  - ✔ The requirements include no unresolved high-risk items.
  - ✔ The requirements won't change much during development.
  - ✔ The team has previous experience with similar projects so that they know what's involved in building the application.
  - ✔ There's enough time to do everything sequentially.

Waterfall Model



In the waterfall model, each step follows the one before in a strict order.

Advantages of using the Waterfall model are as follows:

- Simple and easy to understand and use.
- For smaller projects, the waterfall model works well and yield the appropriate results.
- Since the phases are rigid and precise, one phase is done one at a time, it is easy to maintain.
- The entry and exit criteria are well defined, so it easy and systematic to proceed with quality.
- Results are well documented.

Disadvantages of using Waterfall model:

- Cannot adopt the changes in requirements
- It becomes very difficult to move back to the phase. For example, if the application has now moved to the testing stage and there is a change in requirement, It becomes difficult to go back and change it.
- Delivery of the final product is late
- For bigger and complex projects, this model is not good as a risk factor is higher.
- Not suitable for the projects where requirements are changed frequently.
- Does not work for long and ongoing projects.

Waterfall Variants

- While the basic Waterfall Model is linear, several variants have emerged to address its limitations.

1.Waterfall with Feedback

- In this variant of the traditional Waterfall model, feedback loops are introduced to make the process more flexible.
- In the traditional Waterfall model, each phase flows into the next in a strictly linear sequence.
- However, in the Waterfall with Feedback model, feedback loops are incorporated, allowing revisits to previous phases when issues are discovered later in the process.
- This is especially helpful in identifying and correcting problems early, reducing the risk of large-scale issues down the line.

Waterfall Variants

1.Waterfall with Feedback

- The *waterfall with feedback* variation enables to move backward from one step to the previous step.

If you're working on design and you discover that there was a problem in the requirements, you can briefly go back to the requirements and fix them.

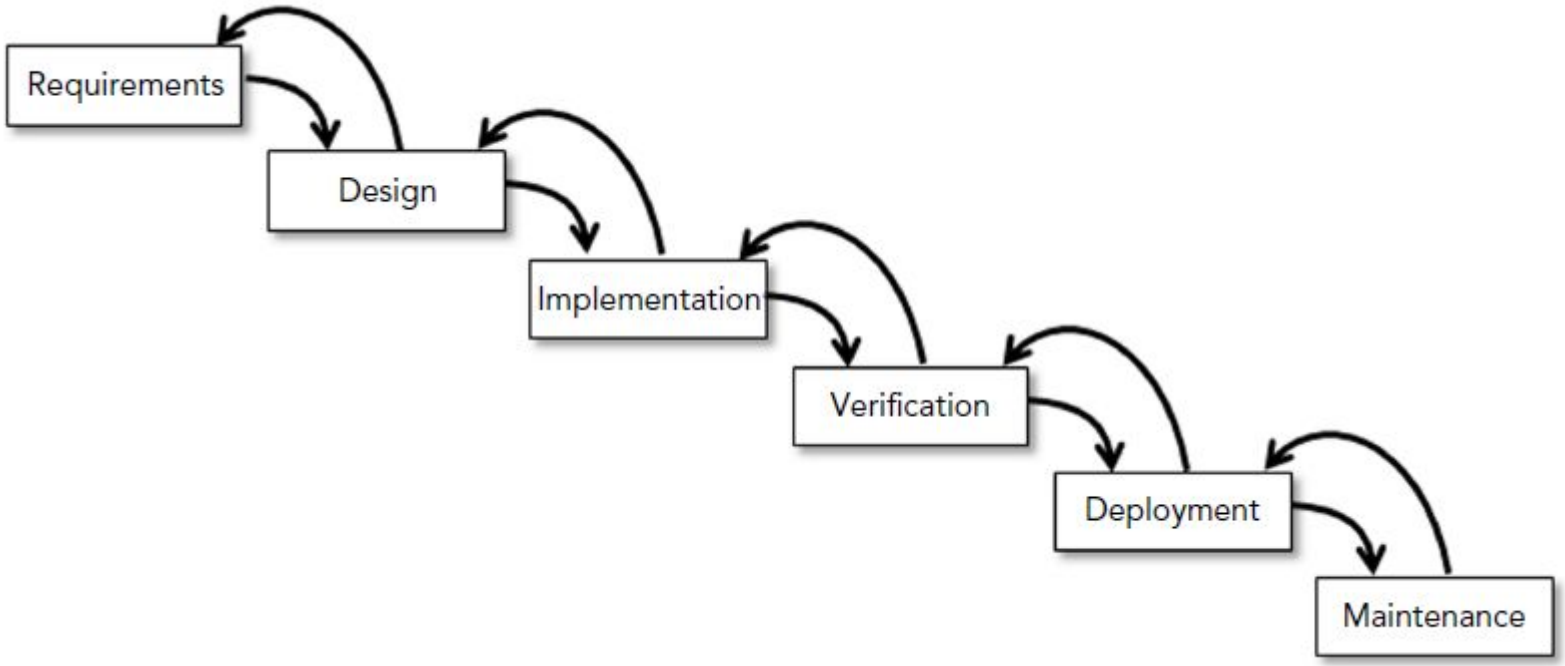
- The farther you have to go back up the cascade, the harder it is.

If you're working on implementation and discover a problem in the requirements, it's hard to skip back up two steps to fix the problem.

- It's also less meaningful to move back up the cascade for the later steps.

If you find a problem during maintenance, then you should probably treat it as a maintenance task instead of moving back into the deployment stage.

Waterfall with Feedback



In the waterfall with feedback model, you can go back to the previous step.

Waterfall with Feedback

Advantages

✔ Feedback Path
✔ Simple
✔ Cost-Effective
✔ Well-organized

Drawbacks

✔ Incremental delivery not supported
✔ Overlapping of phases not supported
✔ Risk handling not supported
✔ Limited customer interactions

Sashimi

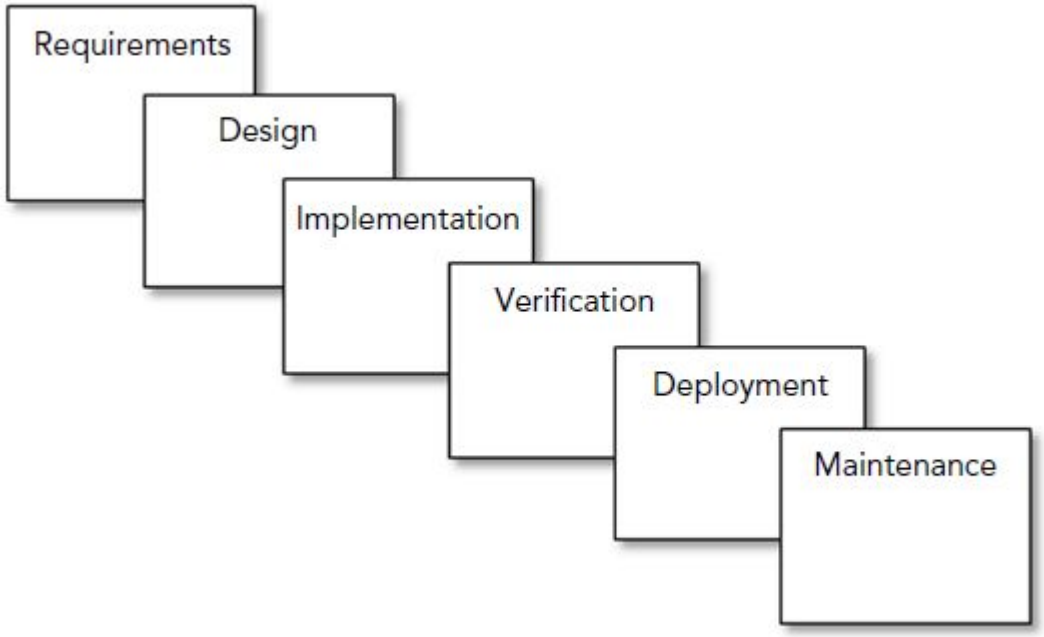
- *Sashimi* (also called *sashimi waterfall* and *waterfall with overlapping phases*) is similar to the waterfall model except the steps are allowed to overlap.
- In a project's first phase, some requirements will be defined while we are still working on others. At that point, some of the team members can start designing the defined features while others continue working on the remaining requirements

Waterfall Variants

2.Sashimi

- The **Sashimi Model**, also known as the "**Waterfall with Overlapping Phases**", is a refined version of the traditional Waterfall software development model.
- *Sashimi* is similar to the waterfall model except the steps are allowed to overlap.
- In a project's first phase, some requirements will be defined while we are still working on others. At that point, some of the team members can start designing the defined features while others continue working on the remaining requirements
- A bit later, the design for some parts of the application will be more or less finished but the design for other parts of the system won't be.
- At that point, some developers can start writing code for the designed parts of the system while others continue working on the rest of the design tasks and possibly even on remaining requirements.

Sashimi



In the sashimi model, development phases overlap.

Sashimi

Advantages of Sashimi Model

- In a project's first phase, some requirements will be defined while you are still working on others.

– For example, some of the team members can start designing the defined features while others continue working on the remaining requirements.

- We can allow greater overlap between project phases.
  - If we have people working on requirements, design, implementation, testing all at the same time.
  - People with different skills can focus on their specialities without waiting for others.
  - It lets you perform a spike or deep dive into a particular topic to learn more about it.
- It lets later phases modify earlier phases.

– If we discover during design that the requirements are impossible or need alterations, we can make the necessary changes.

Sashimi

Disadvantage:

some parts of the application will be more or less finished but the other parts of the system won't be.

Waterfall Variants

3. Incremental Waterfall Model

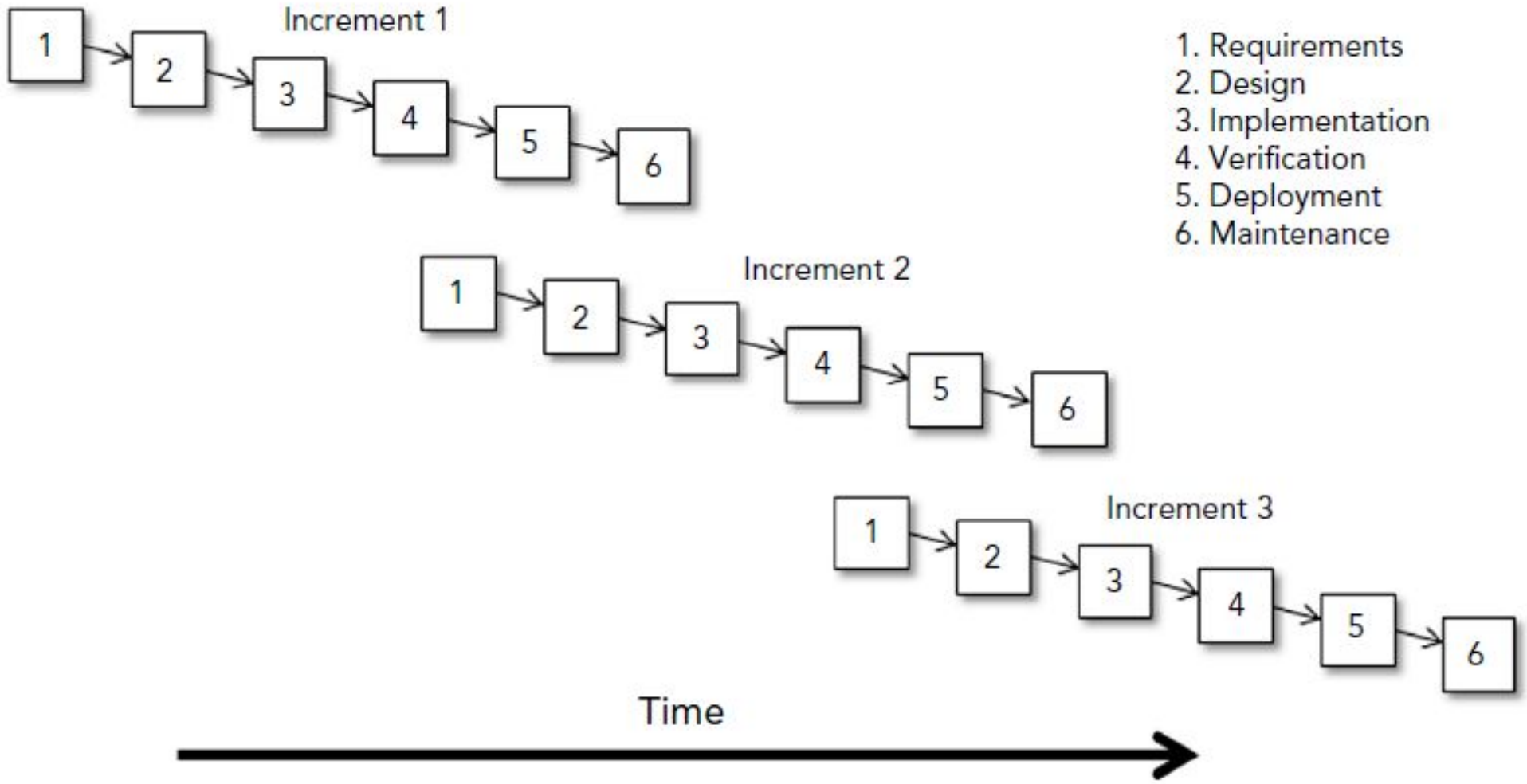
- In this model, the project is divided into smaller, manageable parts (increments), each of which goes through the standard Waterfall phases (requirements, design, development, testing, and deployment).
- Once an increment is delivered, the entire waterfall process restarts for the next increment, building upon the previous one.
- Each increment delivers a functional portion of the final system, allowing for more flexibility and earlier delivery of working software.
- This creates a series of cascading mini-projects, not one continuous flow, allowing for more flexibility and early feedback on smaller parts of the overall product.
- We can change direction when we start a new increment, but within each increment the model runs predictively. In that sense, it is not all that adaptive.

Waterfall Variants

3. Incremental Waterfall Model

- The incremental waterfall model (also called the multi-waterfall model) uses a series of separate waterfall cascades.
- Each cascade ends with the delivery of a usable application called an increment
- Each increment includes more features than the previous one, so you're building the final application incrementally.
- During each increment, you'll get a better understanding of what the final application should look like

Incremental Waterfall Model



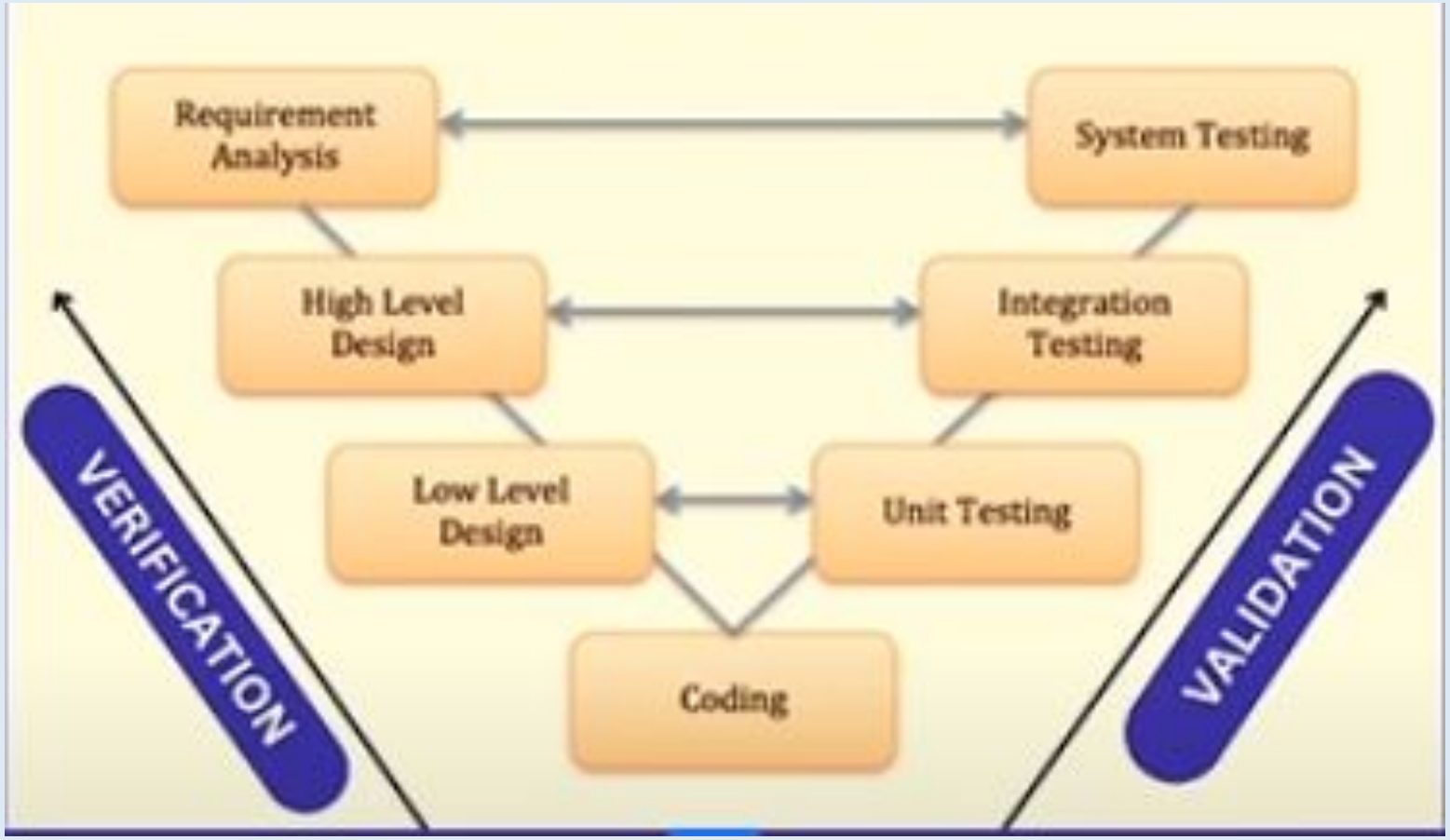
In the incremental waterfall model, a series of waterfall cascades provide an increasing

Waterfall Variants

4.V Model

- The V-Model, or Verification and Validation Model, is a highly disciplined Software Development Life Cycle (SDLC) model and an extension of the Waterfall model.
- *V -model* is basically a waterfall that's been bent into a V shape.
- The tasks on the left side of the V break the application down from its highest conceptual level into more and more detailed tasks.
- This process of breaking the application down into pieces that you can implement is called ***decomposition***
- The tasks on the right side of the V consider the finished application at greater and greater levels of abstraction.
- At the lowest level, testing verifies that the code works.
- At the next level, verification confirms that the application satisfies the requirements, and validation confirms that the application meets the customers' needs.

4.V Model


Requirement Analysis
System Testing
High Level Design
Integration Testing
Low Level Design
Unit Testing
Coding
VERIFICATION
VALIDATION

4.V Model

REQUIREMENT ANALYSIS

The customer's requirements for the software are gathered and analyzed.

HIGH-LEVEL DESIGN

It contains the main modules of the application.

LOW-LEVEL DESIGN

The development team breaks down the system into small modules and specifies the detailed design of each module is called low-level design.

CODING

The development team selects a suitable programming language based on the design and product requirements.

4.V Model

UNIT TESTING

Unit Test Plans are executed to eliminate bugs at code or unit level.

INTEGRATION TESTING

In integration testing, the modules are integrated, and the system is tested. This test verifies the communication of modules among themselves.

SYSTEM TESTING

Test the complete application with its functionality, inter-dependency, and communication.

Software Engineering models

Adaptive Model

- An adaptive development model enables you to change the project's goals if necessary during development.
- An adaptive model lets you periodically reevaluate and decide whether you need to change direction.
- Even then, we cannot say that an adaptive model is always better than a predictive one.
- For example: predictive models work well when the project is relatively small; we know exactly what you need to do, and the time scale is short enough that the requirement won't change during development.
- The adaptive model just gives you chances to fine-tune the project if necessary.

Software Engineering models

Adaptive Model

- Adaptive models embrace the possibility of changing requirements throughout the development process.
- These models focus on iterative development, frequent reassessments, and flexibility.
- 1.Agile Model
- 2.Spiral Model

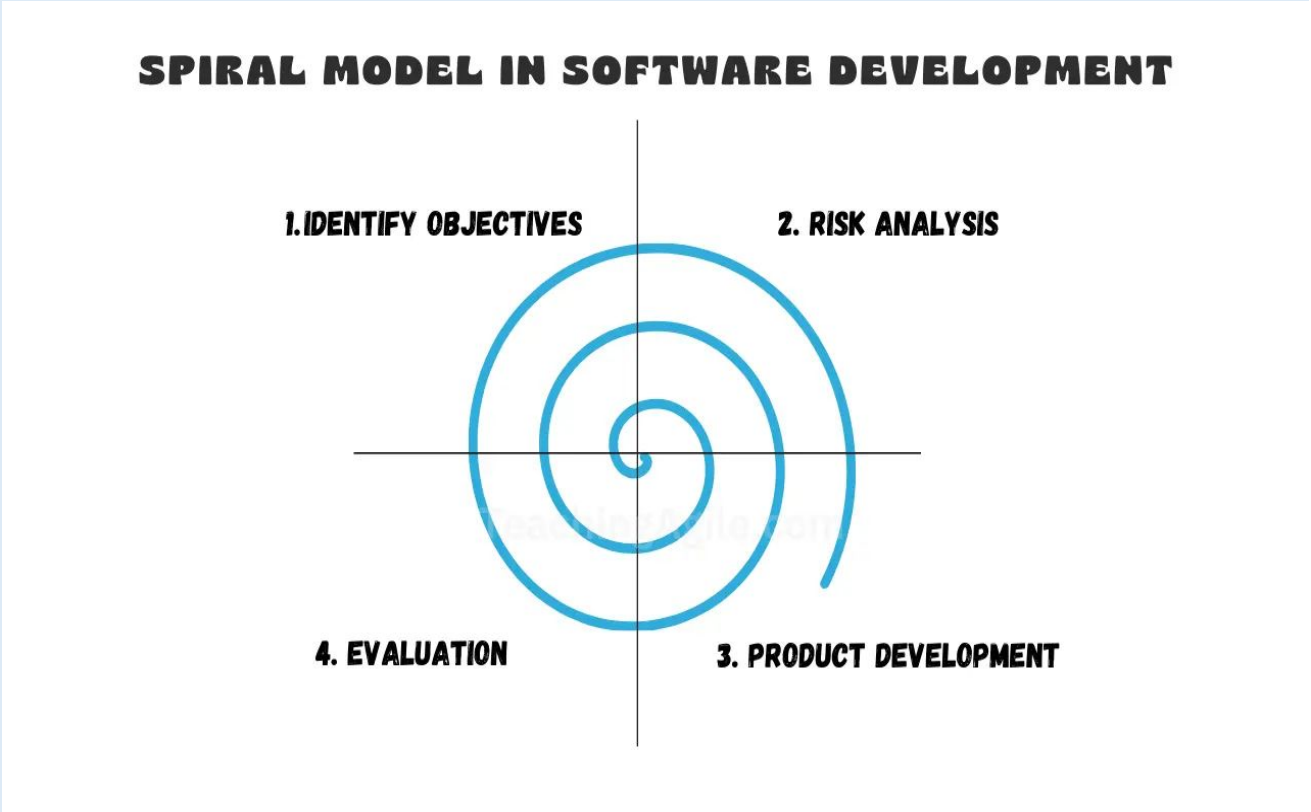
Spiral Model

Spiral model

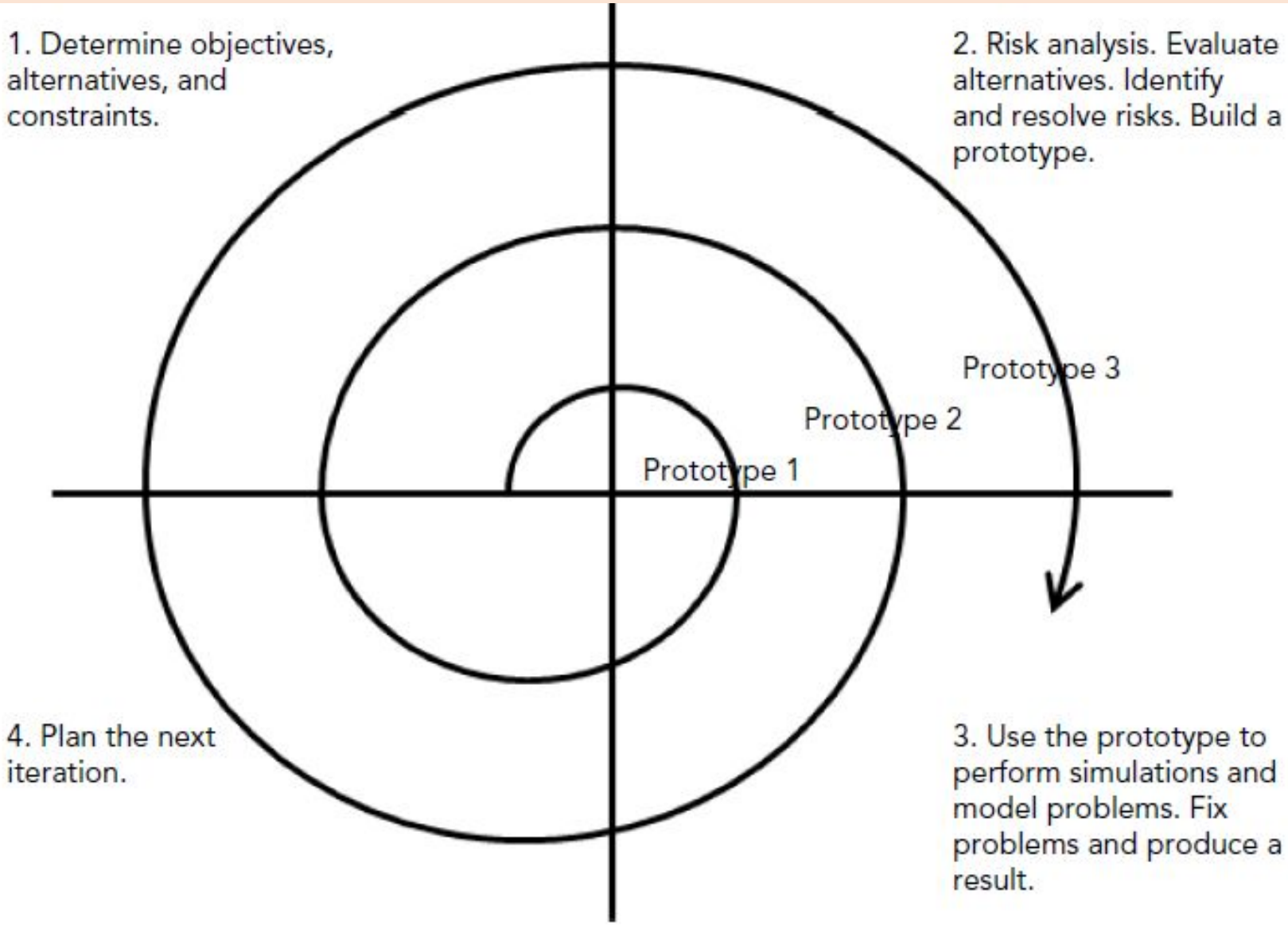
- Initially **proposed by Boehm in 1986.**
- Spiral Model is a **risk-driven software development process model.**
- It is a **combination of Waterfall model, Iterative model, and Prototyping model.**
- Software is developed in a **series of incremental releases as per each spiral.**
- **Example:** Microsoft, OS Versions, Gaming industry, etc.
- Also called as **Meta model.**

Spiral Model is divided into four parts:

1. Planning
2. Risk Analysis
3. Engineering & Execution
4. Customer Evaluation



Spiral Model



Spiral Model

Spiral Model Phases

1. Planning (Requirement Gathering & Analysis)

- Communication between **customers** and **project head**.
- Collect **all the requirements** from customers.
- Analyse **estimated cost, schedule, and required resources**.

2. Risk Analysis

- Identification of **all the potential risks**.
- **Risk mitigation/resolving strategy** is planned for solving risks.
- Design a **prototype** of the model.

Spiral Model

Spiral Model Phases

3. Engineering & Execution

- Actual development starts.
- **Designer** designs the product as per final prototype.
- **Developer** performs actual coding or implementation.
- **Tester** performs all testing methods.
- **Deploy** or release product to the customer environment.

Spiral Model

Spiral Model Phases

4. Evaluation

- Take **feedback** from customers.
- Evaluate the progress so far and make sure the project's major stakeholders agree that the solution came up with is correct and that the project should continue.
- If customer wants any changes, it goes to the next planning phase OR move to the next spiral iteration.

Spiral Model

- If any mistake is made , run another lap around the spiral to fix whatever problems remain.
- Identify the missed objectives, evaluate alternatives, identify and resolve risks, and produce another prototype
- Reaching the right track, plan the next trip around the spiral.

Spiral Model

Advantages of Spiral Model

1. High amount of **risk analysis.**
2. **Risky parts** can be developed earlier, which helps in better risk management.
3. Useful for **large and mission-critical projects.**
4. Allows extensive use of **prototypes.**
5. Always provides space for **customer feedback.**
6. Can accommodate **changing requirements.**
7. **Development is fast.**

Spiral Model

Disadvantages of Spiral Model

1. Risk analysis requires **highly specialized expertise**.
2. It can be a **costly model** to use.
3. Doesn't work well for **smaller projects**.
4. Spiral process is **complex** sometimes because it may go on **indefinitely**.
5. A **large number of spiral stages** requires **excessive documentation**.

Agile Model

- “A flexible, iterative approach to software development”
- It is a combination of iterative and incremental process models.
- The main aim is to help a project to adapt to change requests quickly to facilitate quick project completion by removing unnecessary activities that waste time and effort.
- The requirements are decomposed into many small parts that can be incrementally developed.
- The division of the entire project into smaller parts helps to minimize the project risk and to reduce the overall project delivery time requirements.

Agile Model

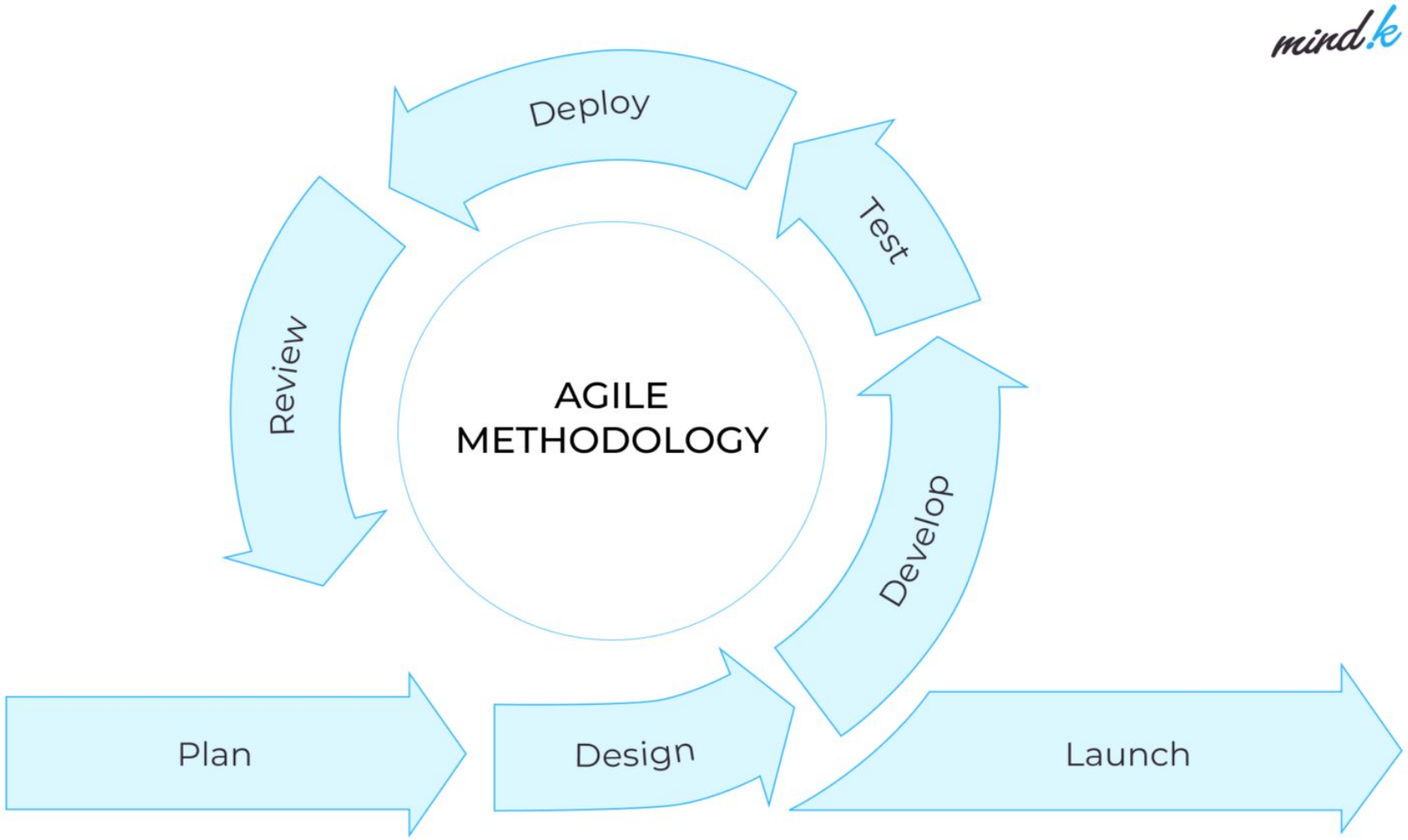
Each iteration are short time frames (timeboxes) that typically last from one to four weeks.

Each iteration includes:

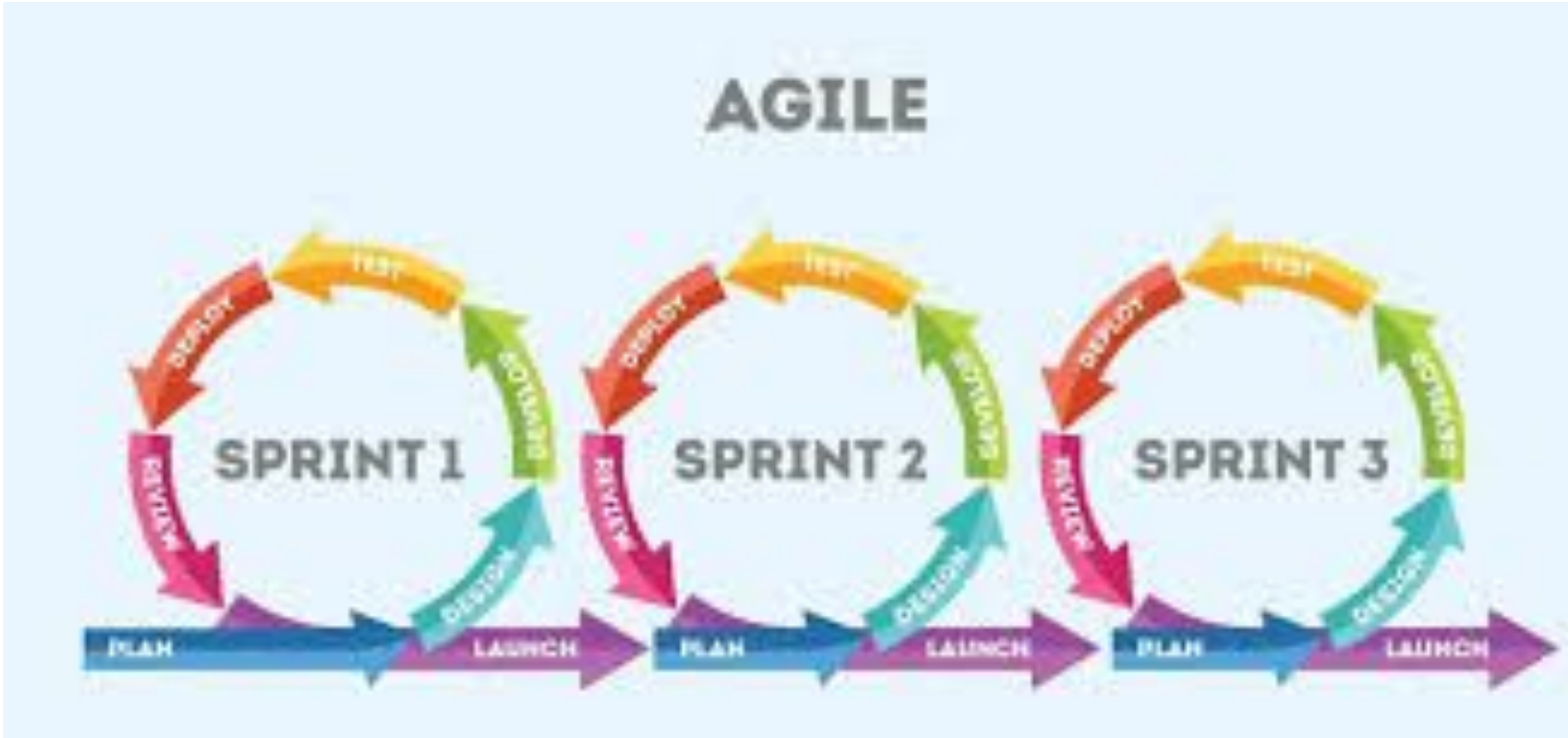
1. Planning
2. Analysis
3. Design
4. Coding
5. Unit Testing
6. Acceptance Testing

End of iteration → product demonstrated to stakeholders.

Agile Model


mind.k
Deploy
Test
Review
AGILE
METHODOLOGY
Develop
Plan
Design
Launch

Agile Model


AGILE
SPRINT 1
SPRINT 2
SPRINT 3
PLAN
LAUNCH
REVIEW
DEPLOY
TEST
DEVELOP
DECISION

- ✔ Each iteration involves a **cross-functional team** working on:

  Planning → Analysis → Design → Coding → Unit Testing → Acceptance Testing
- ✔ At the end of each iteration, a **working product** is demonstrated to stakeholders.
- ✔ Each iteration may not be ready for market release, but should deliver a usable increment with minimal bugs.
- ✔ Multiple iterations may be needed to complete a product or add new features.
- ✔ Agile emphasizes **code development over documentation** and promotes **frequent customer interaction**.

Advantages

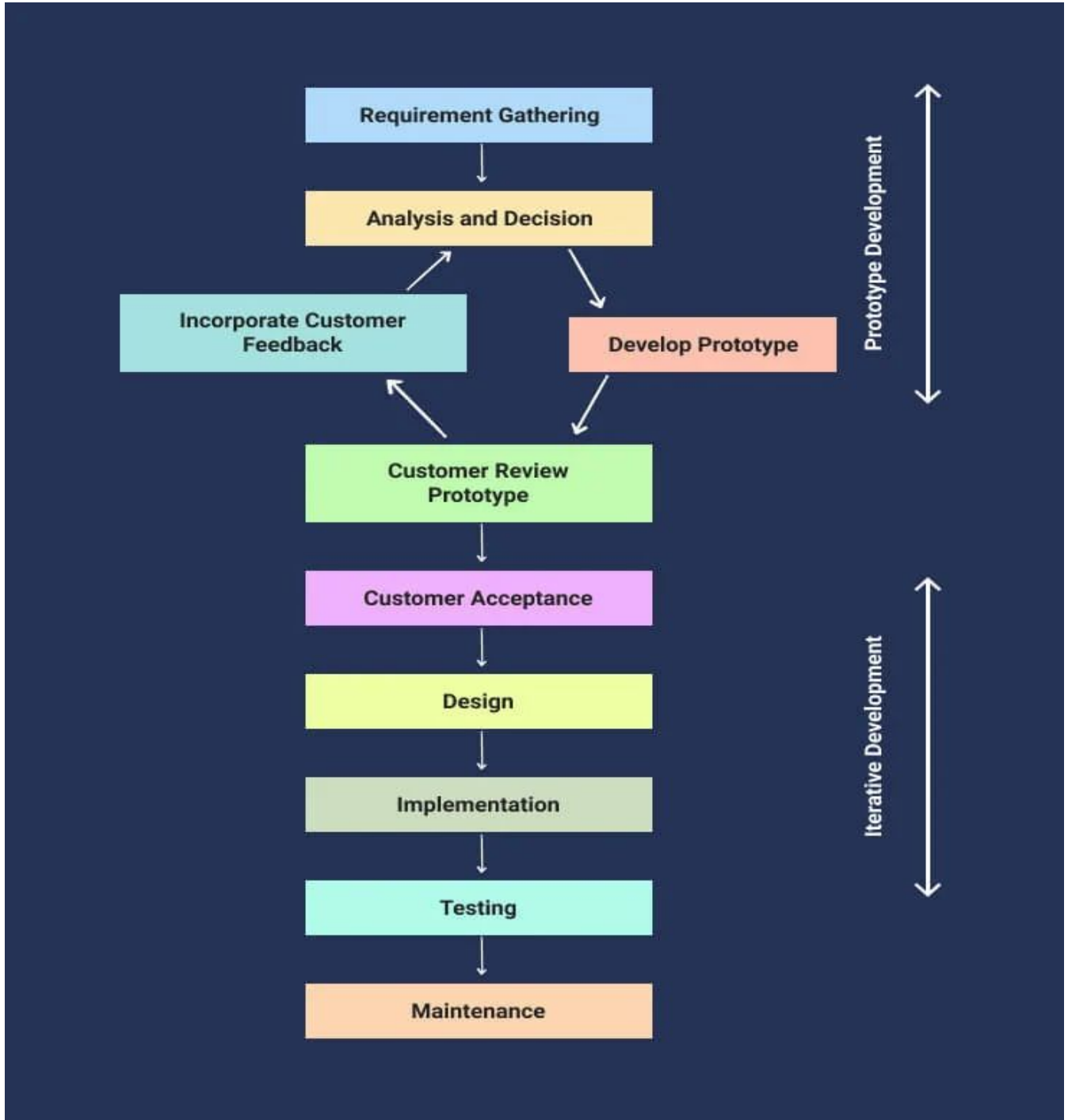
- Faster deployment of software.
- Can quickly adapt to changing requirements and respond faster.
- Strong customer interaction ensures satisfaction in each iteration.
- Immediate feedback helps improve the product in the next iteration.

Disadvantages

- Produces less documentation; more **code-focused**.
- Not suitable for handling **complex dependencies**.
- Depends heavily on **continuous customer interaction**.

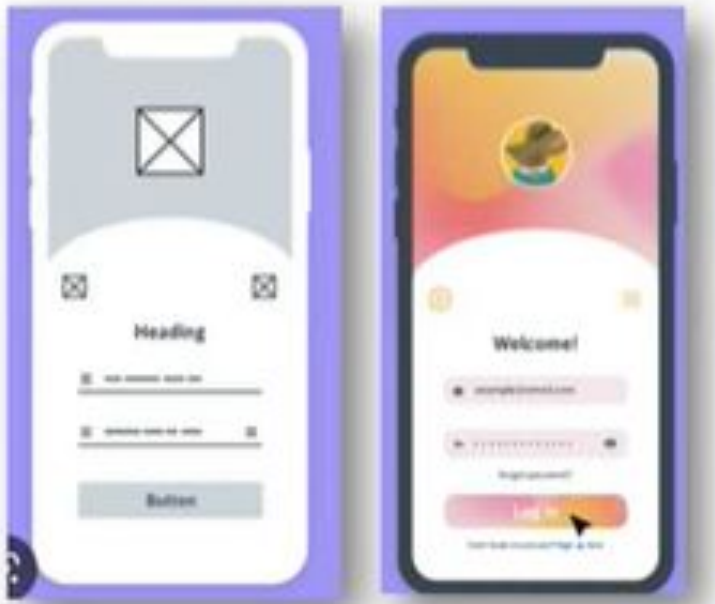
Prototypes

- Prototype is not a complete product; it is just a **functional replica or toy implementation** of any idea, software, or system.
- It is applied **when customers do not know the exact project requirements**.
- It is generated **before actual development** of software.
  - It is an **iterative, trial-and-error method** which takes place between developer and client.

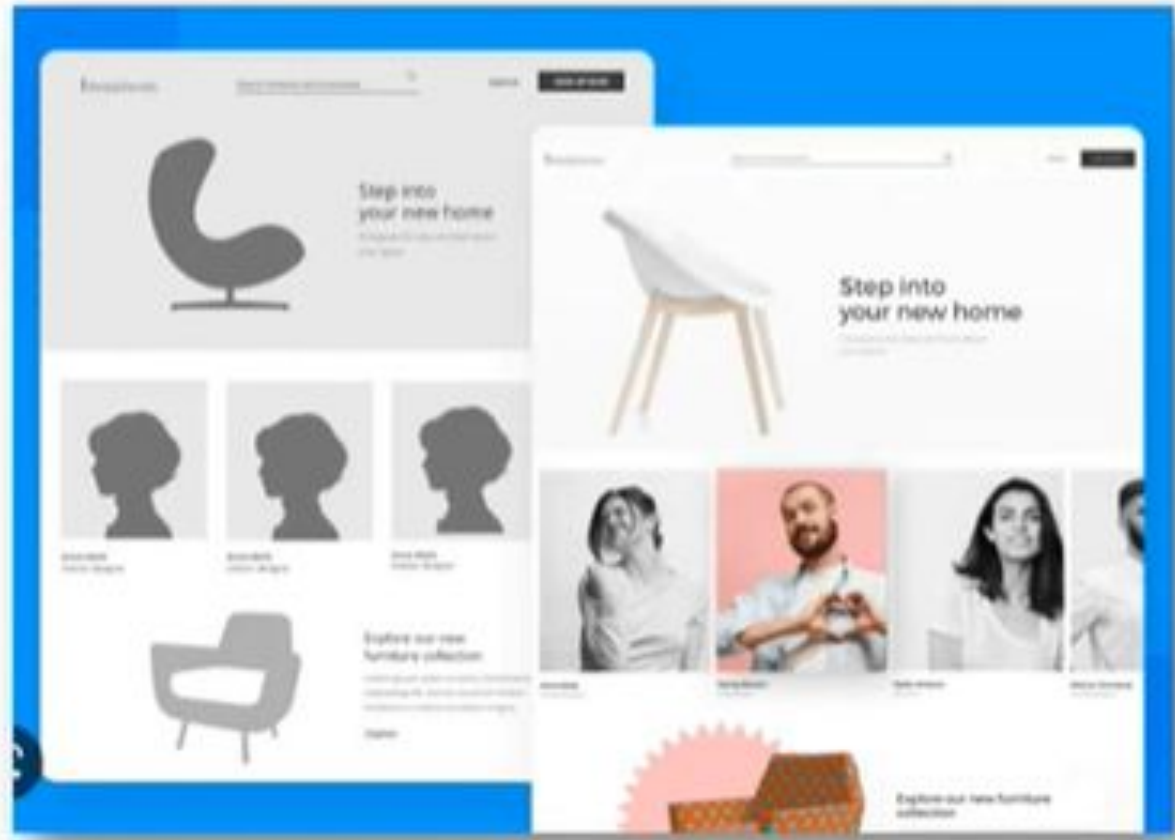

Requirement Gathering
Analysis and Decision
Incorporate Customer Feedback
Develop Prototype
Prototype Development
Customer Review Prototype
Customer Acceptance
Design
Implementation
Testing
Maintenance
Iterative Development

Example Prototype VS Actual

Prototype Actual



Prototype Actual



Prototype Actual

Types of Prototyping Models

Throwaway/Rapid Prototyping:

- **Throwaway Prototyping**, also known as **Rapid Prototyping**, is a software development model where a quick and temporary prototype is built to understand and clarify user requirements.
- Once the prototype has served its purpose, it is discarded, and the actual system is developed based on the refined requirements gathered during the prototyping phase.

Types of Prototyping Models

Evolutionary Prototyping

- **Evolutionary Prototyping** is a software development model where a prototype is gradually refined and developed over time until it evolves into the final system.
- Unlike **Throwaway Prototyping**, where the prototype is discarded after use, in **Evolutionary Prototyping**, the initial prototype is continuously improved based on feedback and additional requirements.
- This model is ideal for projects where requirements are not well-defined from the beginning or are expected to evolve during development.

Types of Prototyping Models

Incremental Prototyping

- **Incremental Prototyping** is a software development model where the system is developed in increments, with each increment being a prototype that focuses on a subset of the overall system's functionality.
- Instead of building a complete prototype of the entire system, smaller, manageable pieces (modules) are developed as separate prototypes.
- These prototypes are developed and tested individually, and once they are complete, they are integrated to form the final system.

Prototypes

Advantages of Prototype Model:

- Users are **actively involved** in the development
- **More accurate user requirements** are obtained.
- **Errors** can be detected much earlier
- **Quicker customer feedback** provides a better idea of customer needs.
- **Missing Functionality** can be identified easily.
- If customer not satisfied with prototype than we can develop a new prototype.

Disadvantages of Prototype Model:

- If the user is **not satisfied** by the developed prototype, then a new prototype is developed. This process goes on until a perfect prototype is developed. Thus, this model is **time consuming and expensive**.
- After seeing an early prototype the users may **demand the actual system to be delivered in soon**.
- If end-user not satisfied with initial prototype, he/she may **lose interest in the project**.